Multimedia Application

By

Minhaz Uddin Ahmed, PhD

Department of Computer Engineering

Inha University Tashkent.

Email: minhaz.ahmed@gmail.com

Content

- Language Models
- N-Grams
- 3.2 Evaluating Language Models: Training and Test Sets
- 3.3 Evaluating Language Models: Perplexity
- 3.4 Sampling sentences from a language model
- 3.5 Generalization and Zeros
- 3.6 Smoothing
- 3.8 Advanced: Kneser-Ney Smoothing

Language Modeling

- Language modeling involves predicting the probability distribution of words or tokens in a sequence of text. The goal of language modeling is to capture the underlying structure and patterns of natural language, allowing computers to generate coherent and grammatically correct text.

- There are several approaches to language modeling, including:

i) N-gram Models

ii) Neural Network Models

iii) Transformer Models

Language Modeling

- Tashkent is the capital of ----------?

  i) India

  ii) China

  iii) Uzbekistan

Language model applications

- Spell checking
- Grammer Checking
- Machine translation
- Summarization
- Question answering
- Speech recognition

Probabilistic Language Models

- Assign a probability to a sentence

Application:

- Machine Translation:

  P(high winds tonite) > P(large winds tonite)

- Spell Correction
  - The office is about fifteen **minuets** from my house
    - P(about fifteen **minutes** from) > P(about fifteen **minuets** from)
- Speech Recognition
  - P(I saw a van) >> P(eyes awe of an)
- + Summarization, question-answering,

Probability of sentence

- Grammer correction
- I go to school
- I going to school

- Probability score: I go to school > I going to school

- Correct: go to school, Wrong: going to school

Probability of sentence or words

- Compute the probability of a sentence or sequence of words:

  => P(W) = P(w1, w2,w3, w4,w5…wn)

- Probability of an upcoming word:

  => P(w5| w1,w2,w3,w4)

- P(Uzbekistan | Tashkent , is, the, capital, of)

- A model that computes either of these :

  P(W) or P(wn|w1, w2…wn-1) is called a language model.

How to compute P(W)

- How to compute this joint probability:
  - P(its, water, is, so, transparent, that)
- Intuition: let's rely on the Chain Rule of Probability

$$P(A,B) = p(A|B)\, p(B)$$

We can extend this for three variables:

$$P(A,B,C) = P(A|\, B,C)\, P(B,C) = P(A|B,C)\, P(B|C)\, P(C)$$

and in general to n variables:

$$P(A1, A2, ..., An) = P(A1|\, A2, ..., An)\, P(A2|\, A3, ..., An)\, P(An\text{-}1|An)\, P(An)$$

In general we refer to this as the *chain rule*

the joint probability of all the random variables can be calculated by multiplying the probability of each variable conditioned on all the previous variables

Chain Rule of Probability

- Conditional probabilities

  => $P(B|A) = P(A,B) / P(A)$

  Rewriting : $P(A,B) = P(A)P(B|A)$

More variables: $P(A,B,C,D) = P(A)\ P(B|A)\ P\ (C|A,\ B)\ P(D|A,B,C)$

The chain rule in general

=> $P(x1,\ x2,\ x3,\ \ldots,\ xn) = P(x1)\ P(x2|x1)\ P(x3|x1,x2)\ \ldots\ P(xn|x1,\ \ldots,\ xn\text{-}1)$

Chain Rule of Probability

- Chain rule : $P(A,B,C,D) = P(A)\ P(B|A)\ P(C|A,B)\ P(D|A,B,C)$

- Example

  = P(Tashkent is the capital of Uzbekistan)

  = P(Tashkent) x P(is | Tashkent) x P(the | Tashkent, is) x P(capital | Tashkent, is the) x P(of | Tashkent, is, the, capital) x P(Uzbekistan | Tashkent, is, the, capital, of)

Chain Rule of Probability

- Example

= P (Tashkent is the capital of Uzbekistan)

= P(Tashkent) X P(is |Tashkent)X P(the| Tashkent, is) x P(capital | Tashkent, is, the) X P(of | Tashkent, is, the, capital) x P(Uzbekistan| Tashkent, is ,the, capital, of)

Calculation

= P(Uzbekistan| Tashkent, is ,the, capital, of)

= count (Tashkent is the capital of Uzbekistan) / count (Tashkent is the capital of)

The Chain Rule applied to compute joint probability of words in sentence

$$P(w_1 w_2 \cdots w_n) = \prod_i P(w_i \mid w_1 w_2 \cdots w_{i-1})$$

P("its water is so transparent") =

P(its) × P(water|its) × P(is|its water)

× P(so|its water is) × P(transparent|its water is so)

How to estimate these probabilities

- Could we just count and divide?

$$P(\text{the} \mid \text{its water is so transparent that}) = \frac{Count(\text{its water is so transparent that the})}{Count(\text{its water is so transparent that})}$$

- No! Too many possible sentences!
- We'll never see enough data for estimating these

Markov Assumption

- Simplifying assumption

  = P(Uzbekistan| Tashkent, is, the, capital, of)

  = P(Uzbekistan | of)

  = P (Uzbekistan | capital of)

Andrei Markov

The assumption that the probability of a word depends only on the previous word is called Markov assumption

Simplest case: Unigram model

$$P(w_1 w_2 \ldots w_n) \approx \prod_i P(w_i)$$

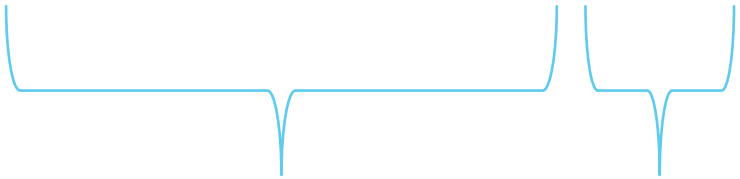
Some automatically generated sentences from a unigram model

```
fifth, an, of, futures, the, an, incorporated, a,
a, the, inflation, most, dollars, quarter, in, is,
mass

thrift, did, eighty, said, hard, 'm, july, bullish

that, or, limited, the
```

Bigram model

- Condition on previous word

- Please bring me a glass of water.



History

Word prediction

Estimating bigram probabilities

- The Maximum Likelihood Estimate

$$P(w_i \mid w_{i-1}) = \frac{count(w_{i-1}, w_i)}{count(w_{i-1})}$$

$$P(w_i \mid w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

Bigram model

<s> I am Sam </s>

<s> Sam I am </s>

<s> I do not like green eggs and ham </s>

$$P(w_i \mid w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

$P(\texttt{I} \mid \texttt{<s>}) = \frac{2}{3} = .67$ $P(\texttt{Sam} \mid \texttt{<s>}) = \frac{1}{3} = .33$ $P(\texttt{am} \mid \texttt{I}) = \frac{2}{3} = .67$

$P(\texttt{</s>} \mid \texttt{Sam}) = \frac{1}{2} = 0.5$ $P(\texttt{Sam} \mid \texttt{am}) = \frac{1}{2} = .5$ $P(\texttt{do} \mid \texttt{I}) = \frac{1}{3} = .33$

Estimated bigram probabilities

- P(<s> I want English food </s>) = P(I|<s>)x P (want|I)x P(English|want) x p(food|english) x P(</s>|food) = 0.000031

- Given that

P(I|<s>) = 0.25

P (want|I)= 0.33

P(English|want)= 0.0011

p(food|english)=0.5

P(</s>|food) = 0.68

$$P(w_i \mid w_{i-1}) = \frac{count(w_{i-1}, w_i)}{count(w_{i-1})}$$

N-gram models

- We can extend to trigrams, 4-grams, 5-grams
- In general this is an insufficient model of language
  - because language has **long-distance dependencies**:

  "The computer which I had just put into the machine room on the fifth floor crashed."

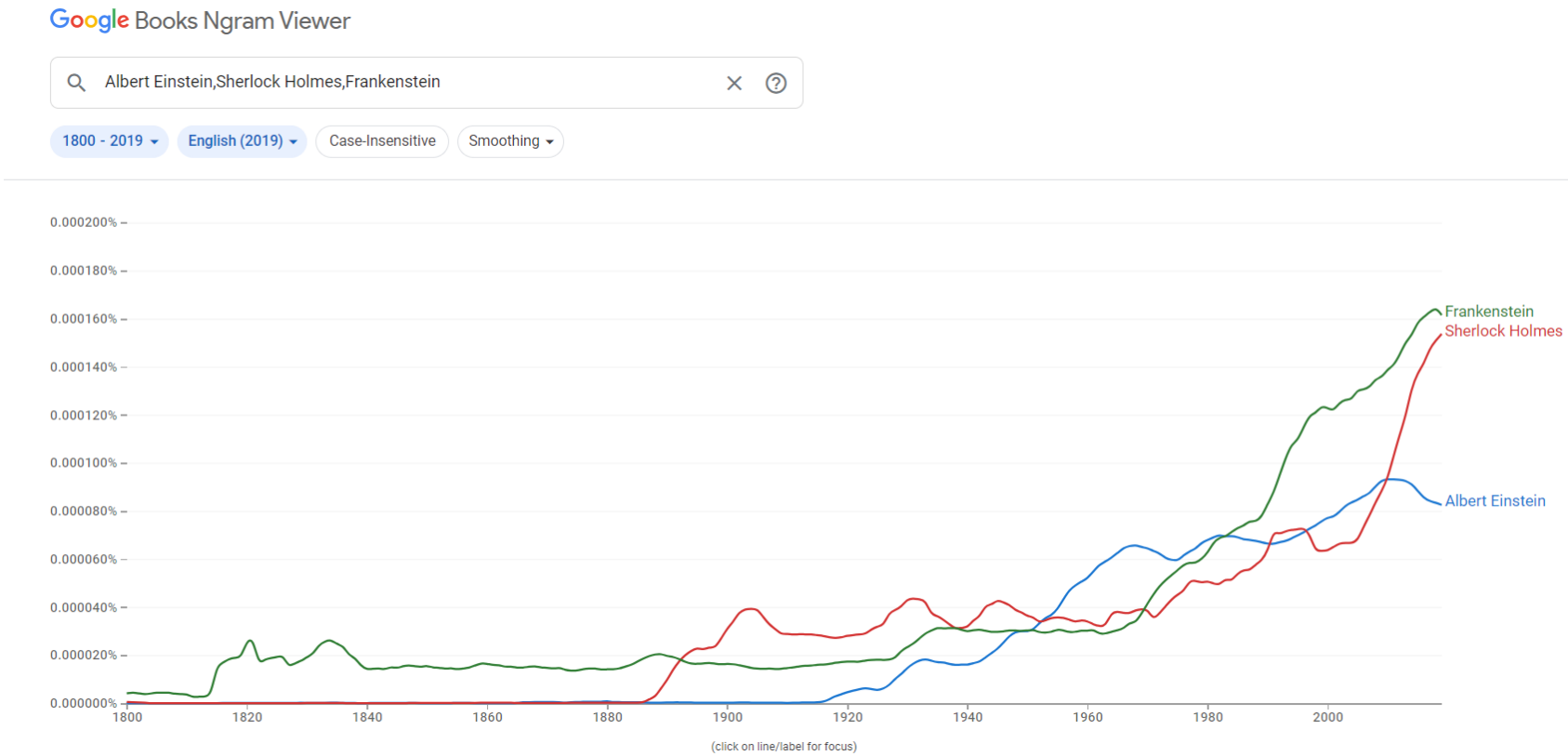
- But we can often get away with N-gram models

N-gram models

- An n-gram is a collection of n successive items in a text document that may include words, numbers, symbols, and punctuation. N-gram models are useful in many text analytics applications where sequences of words are relevant, such as in sentiment analysis, text classification, and text generation.

- In deep learning , Language models used higher gram model to train the dataset.

N-gram models



Google Ngram Viewer displays user-selected words or phrases (ngrams) in a graph that shows how those phrases have occurred in a corpus. Google Ngram Viewer's corpus is made up of the scanned books available in Google Book

Once the language model is built, it can then be used with machine learning algorithms to build predictive models for text analytics applications

Google N-Gram Release, August 2006

AUG
3

All Our N-gram are Belong to You

Posted by Alex Franz and Thorsten Brants, Google Machine Translation Team

Here at Google Research we have been using word n-gram models for a variety of R&D projects,

...

That's why we decided to share this enormous dataset with everyone. We processed 1,024,908,267,229 words of running text and are publishing the counts for all 1,176,470,663 five-word sequences that appear at least 40 times. There are 13,588,391 unique words, after discarding words that appear less than 200 times.

Evaluating Language Models: Training and Test Sets

- "Extrinsic (in-vivo) Evaluation"

To compare models A and B

1. Put each model in a real task
    - Machine Translation, speech recognition, etc.
2. Run the task, get a score for A and for B
    - How many words translated correctly
    - How many words transcribed correctly
3. Compare accuracy for A and B

Intrinsic (in-vitro) evaluation

- Extrinsic evaluation not always possible
  - Expensive, time-consuming
  - Doesn't always generalize to other applications
- Intrinsic evaluation: **perplexity**
  - Directly measures language model performance at predicting words.
  - Doesn't necessarily correspond with real application performance
  - But gives us a single general metric for language models
  - Useful for large language models (LLMs) as well as n-grams

Training sets and test sets

We train parameters of our model on a **training set**.

We test the model's performance on data we haven't seen.

- A **test set** is an unseen dataset; different from training set.
  - Intuition: we want to measure generalization to unseen data
- An **evaluation metric** (like **perplexity**) tells us how well our model does on the test set.

Perplexity

- Perplexity is the standard metric for measuring quality of a language model.
- The inverse probability of test set, normalized by the number of words.

$$PP(W) = P(w_1 w_2 \ldots w_N)^{-\frac{1}{N}}$$

$$= \sqrt[N]{\frac{1}{P(w_1 w_2 \ldots w_N)}}$$

Chain rule:

$$\mathrm{PP}(W) = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i | w_1 \ldots w_{i-1})}}$$

Bigrams:

$$\mathrm{PP}(W) = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i | w_{i-1})}}$$

Minimizing perplexity is the maximizing probability

Perplexity

- Calculate perplexity of a sentence

Task of recognizing the digit in English
=> A sentence consist of random digits
=> Each digit probability is p = 1/10

$$
\begin{aligned}
\mathrm{PP}(W) &= P(w_1 w_2 \ldots w_N)^{-\frac{1}{N}} \\
&= \left(\frac{1}{10}^{N}\right)^{-\frac{1}{N}} \\
&= \frac{1}{10}^{-1} \\
&= 10
\end{aligned}
$$

Minimizing perplexity is the maximizing probability

Choosing training and test sets

- If we're building an LM for a specific task
  - The test set should reflect the task language we want to use the model for
- If we're building a general-purpose model
  - We'll need lots of different kinds of training data
  - We don't want the training set or the test set to be just from one domain or author or language.

Training on the test set

We can't allow test sentences into the training set

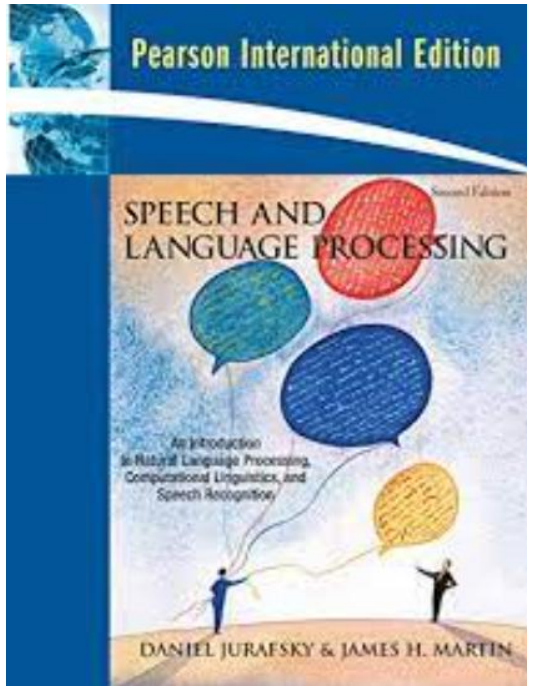
- Or else the LM will assign that sentence an artificially high probability when we see it in the test set
- And hence assign the whole test set a falsely high probability.
- Making the LM look better than it really is

This is called "Training on the test set"

Dev sets

- If we test on the test set many times we might implicitly tune to its characteristics
  - Noticing which changes make the model better.
- So we run on the test set only once, or a few times
- That means we need a third dataset:
  - A **development test set** or, **devset**.
  - We test our LM on the devset until the very end
  - And then test our LM on the **test set** once

Reference



Chapter 3

Question

Thank you